CrossCheck: Model Comparison for Real-Time Misinformation Detection

IMCS 3020U Final Report

CrossCheck Website: `https://crosscheckweb.vercel.app`

GitHub Repo: `https://github.com/RyanCoones/IMCS3020U-Project`

Ryan Coones, Jeremy McPaul, Lucas Fischer

April 25, 2026

Contents

1 Introduction

Approximately 402.74 million terabytes of data are generated on the internet every single day, according to data sourced from February 2026 [1]. An astounding amount of information is shared on the internet, and with the majority (56%) of American adults admitting to consuming their news digitally, it is essential that we carefully consider the credibility of the information we consume online [2]. The goal of CrossCheck is to inform users when the content of an article is questionable. In this context, "questionable" encompasses articles that may be factually incorrect, that present biased or opinionated language as objective reporting, or that misrepresent information through misleading framing.

CrossCheck achieves this through two components: a browser extension and a synchronized web application powered by Amazon Web Services that generates a credibility score, and backs the score through AI-powered writing analysis and automated fact-checking. Over the course of this project, we explored and compared several classification models including Bernoulli Naive Bayes, RNN, LSTM, BiLSTM, and GRU, before implementing a stacked generalization ensemble that combines the strengths of multiple classifiers through a meta-learning approach. The WELFake Fake News dataset from Kaggle was used to train all our models, and contains a near even split of 72134 fake/real classified articles. The dataset was merged from 4 smaller news datasets (Kaggle, McIntire, Reuters, and BuzzFeed Political), and most of the articles were sourced from 2016–2017 [3].

This report covers the background and motivation behind our model selection, the methodology used for training, evaluation, deployment, the results and error analysis from our experiments, and plans for future development.

2 Background

In this section, we review the classification models we experimented with, starting from the simplest and increasing in complexity. This approach allowed us to establish a baseline with Bernoulli Naive Bayes before exploring recurrent architectures that better capture sequential context. We also describe the embedding process used to prepare text data for the neural models.

2.1 Classification Models

2.1.1 Bernoulli Naive Bayes

The Bernoulli Naive Bayes (BNB) Model is a probabilistic model that leverages Bayes Theorem to make classifications. Bayes theorem is given as:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)},$$

which refers to the probability of an event A occurring, given that event B has occurred. The model is named "Naive" because it treats the presence of all features (words) in a document as independent events. However, the presence of each word within a sentence will obviously depend on the context of the sentence. This makes Naive Bayes models less effective for classification, although, it is consequently highly efficient [4]. In this project, we are using Bernoulli Naive Bayes, which is best suited for binary features. We compute the probability of an article being real/fake while applying Laplace smoothing to prevent 0 probabilities. Generalization of learned probabilities is performed through joint log likelihoods, which gives the final probability of the article being real/fake [5].

2.1.2 Recurrent Neural Network

RNNs were designed to analyze sequences of data, rather than individual data points. It features a feedback loop, which allows the model to consider previous context when making a prediction. Figure 1 is an architectural diagram of an RNN.

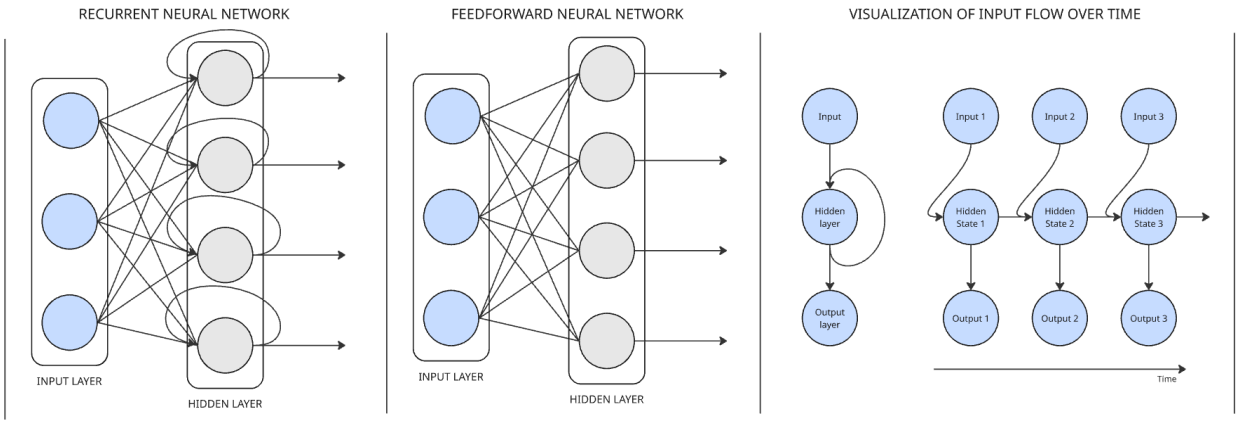


Figure 1: RNN architectural diagram and comparison with feedforward neural network

RNNs would address the shortcomings of the BNB model, since it does not treat features

as independent instead, it learns lexical patterns. This would greatly contribute to the classification of text. Unfortunately, RNNs have a problem with exploding and vanishing gradients due to sigmoid and ReLU activation functions. This impacts their ability to retain important information, which makes them perform poorly [6].

2.1.3 Long Short Term Memory Classifier

The LSTM is an enhanced RNN, which was created to address the vanishing and exploding gradients problem. It introduces mechanisms to adjust the hidden state and the maintained cell state at each time step. For the LSTM, this includes the "forget" gate, the "input" gate, and the "output" gate. Figure 2 is an architectural diagram of an "LSTM cell", which is a visualization of the computations being performed at each time step.

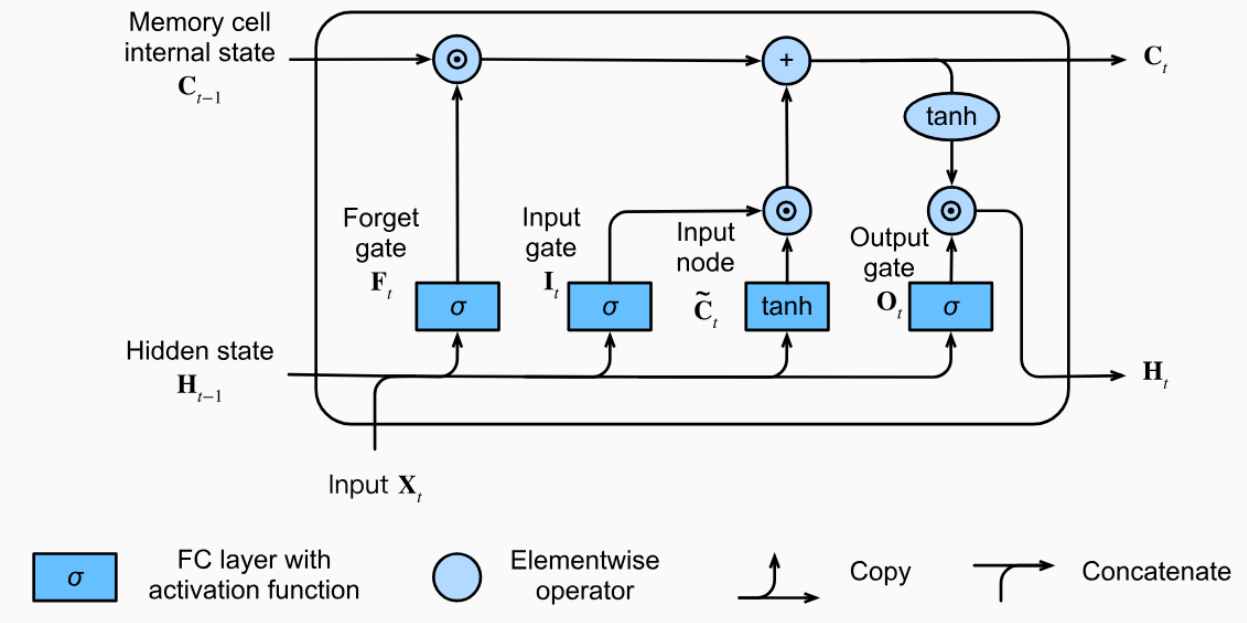


Figure 2: Architectural diagram of an LSTM cell

There is also an extension of the LSTM, called the Bidirectional LSTM (BiLSTM) that feeds forward and backward to capture context from previous and future data. The BiLSTM is much better for NLP tasks, but has a tendency to overfit on small datasets due to the number of tunable parameters being doubled. Figure 3 illustrates the function of a BiLSTM.

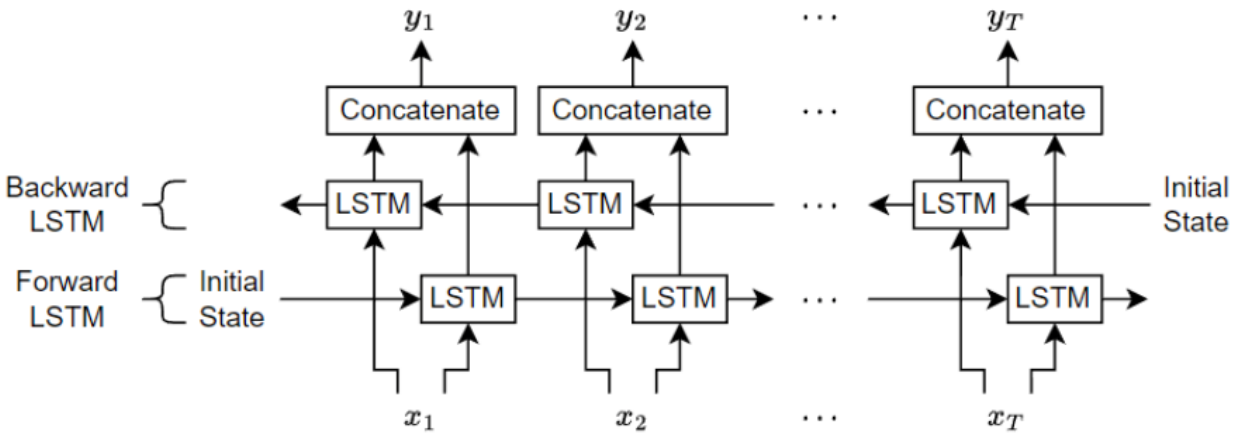


Figure 3: Architectural diagram of a BiLSTM neural network

2.1.4 Gated Recurrent Units Network

The GRU cell works very similarly to the LSTM cell, but the GRU cell consists of only two gates, which makes it much more efficient. GRU has a reset gate and an update gate [7]. Figure 4 depicts the architecture of a GRU cell.

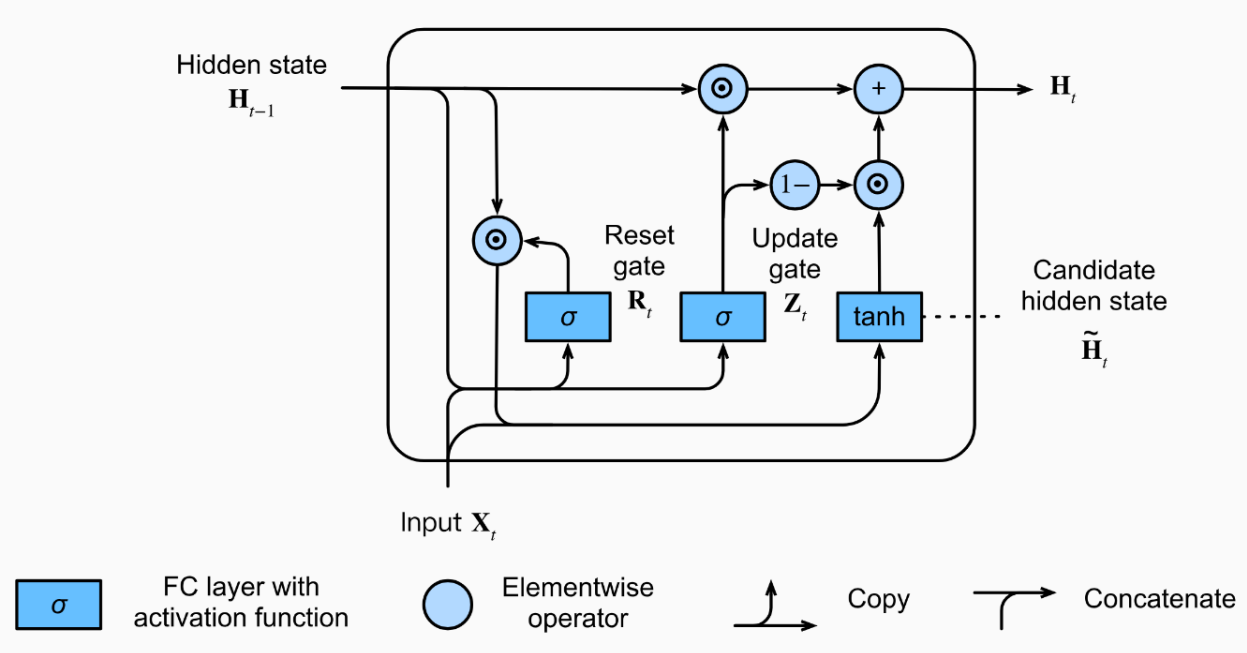
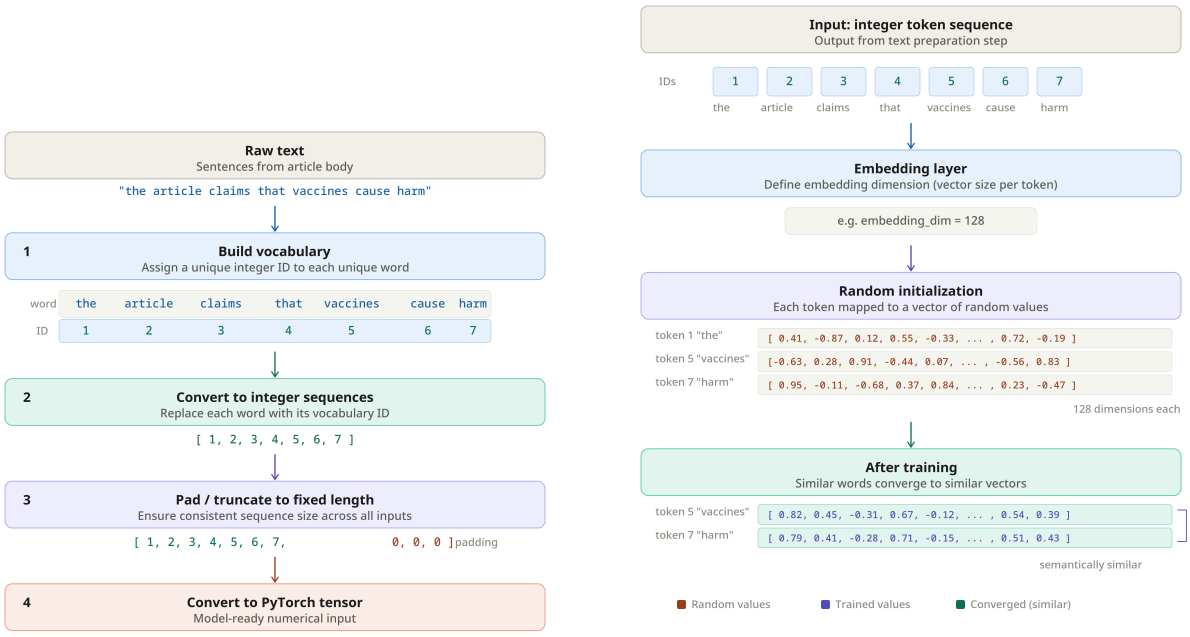


Figure 4: Architectural diagram of a GRU cell [7]

2.2 Embedding Process for Neural Models

The process of creating embeddings is critical. Embeddings are numerical representations of words, digestible to neural models. Figure 5 is a flow chart, showing the text preprocessing and embedding process CrossCheck was built upon.



(a) Text preprocessing

(b) Embedding creation

Figure 5: Text to tensor conversion and creation of embeddings

Before any text reaches a model, it must be transformed into a fixed-length numerical sequence. Raw text from articles is normalized and cleaned, then each word is mapped to an integer based on a pre-built vocabulary. We limited the vocabulary to 10000 words, as this captures the vast majority of meaningful tokens while filtering out rare words that appear too infrequently to aid generalization. Sequences are padded or truncated to 400 tokens based on the distribution of article lengths in the dataset, and converted to PyTorch tensors.

The embedding layer, which is the first layer of each neural model, assigns each token a dense vector of learned values. These vectors are initially random, but as the model trains, words appearing in similar contexts converge to similar representations, allowing the model to capture semantic relationships without explicit linguistic rules [8].

3 Development & Results

3.1 Stacked Generalization

To address the limitations of relying on a single classifier, we implemented a stacked generalization ensemble. Stacking combines the outputs of multiple base models by training a meta-learner to determine how much to trust each prediction [9]. Our ensemble uses three base models, GRU, LSTM, and BNB, each producing a probability score indicating the likelihood that an article is fake. The GRU and LSTM models are architecturally similar, but the BNB is a probabilistic model whose behaviour is fundamentally different. When BNB disagrees with the neural networks, it reveals an informative pattern about the data.

The meta-learner must be trained on predictions that the base models never saw during their own training. otherwise, it simply learns to trust whichever base model overfits the most. To generate these predictions, we use k-fold cross-prediction [10]. For each base model, we partition the training data into k mutually exclusive folds. Each model is trained k times, each time on $k - 1$ folds, and makes predictions on the one remaining held-out fold. Stitching together all k held-out predictions produces a full-length out-of-fold (OOF) prediction vector. Once all three base models have produced their OOF vectors, we stack them alongside the standard deviation and spread between predictions to form the MLP's training set. The MLP learns which combinations of these features are most predictive, producing a single final probability and classification label. Figure 6 illustrates this architecture.

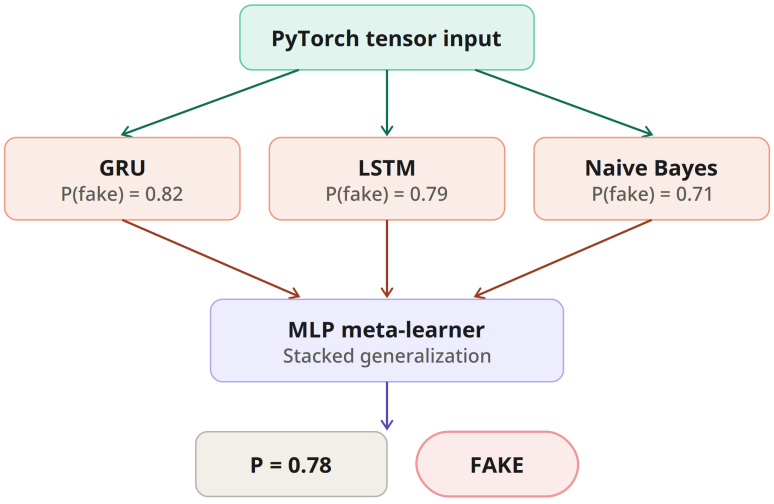


Figure 6: Ensemble classification with MLP meta-learner

3.2 Deployment Pipeline

The deployment pipeline enables real-time classification through a Chrome browser extension and a synchronized web application. When a user clicks "Check Current Page," the extension sends the current tab URL to a FastAPI backend hosted on Railway. The API extracts article content using trafilatura, with newspaper4k as a fallback, and applies the same preprocessing pipeline used during training. The cleaned text is passed through the ensemble classifier, which produces a probability score and a real/fake label. The result is stored in an AWS RDS database. The extension displays the results, while the web application, built with React and Node.js and hosted on Vercel, retrieves the result for future reference.

Figure 7 shows the deployment pipeline, from URL input to the final result displayed in the browser extension and web app.

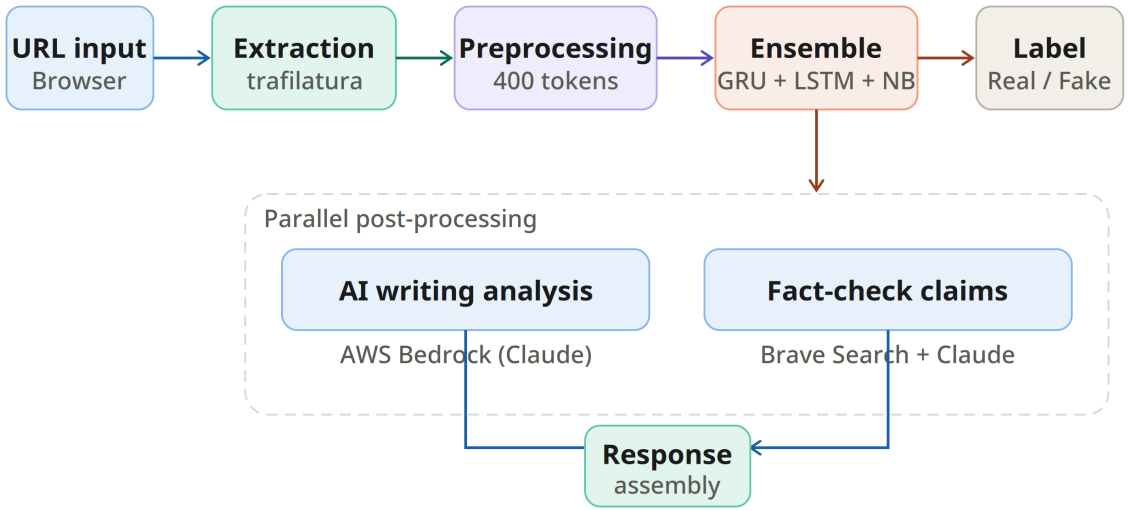


Figure 7: End-to-end deployment pipeline

After classification, two more tasks run in parallel. The first sends the article text to AWS Bedrock, where Claude performs a style analysis and returns a brief summary of stylistic concerns. The second sends the text to an AWS Lambda function, where Claude extracts verifiable claims from the article. Each claim is searched against the web using the Brave Search API, and Claude compiles the results into verdicts such as supported, contradicted, or unverified. These outputs are assembled alongside the classification result into the final response object, which the web application presents with a concern level badge,

an AI analysis section, and individual fact-check verdict cards.

Figure 8 outlines the two additional processes that run after classification.

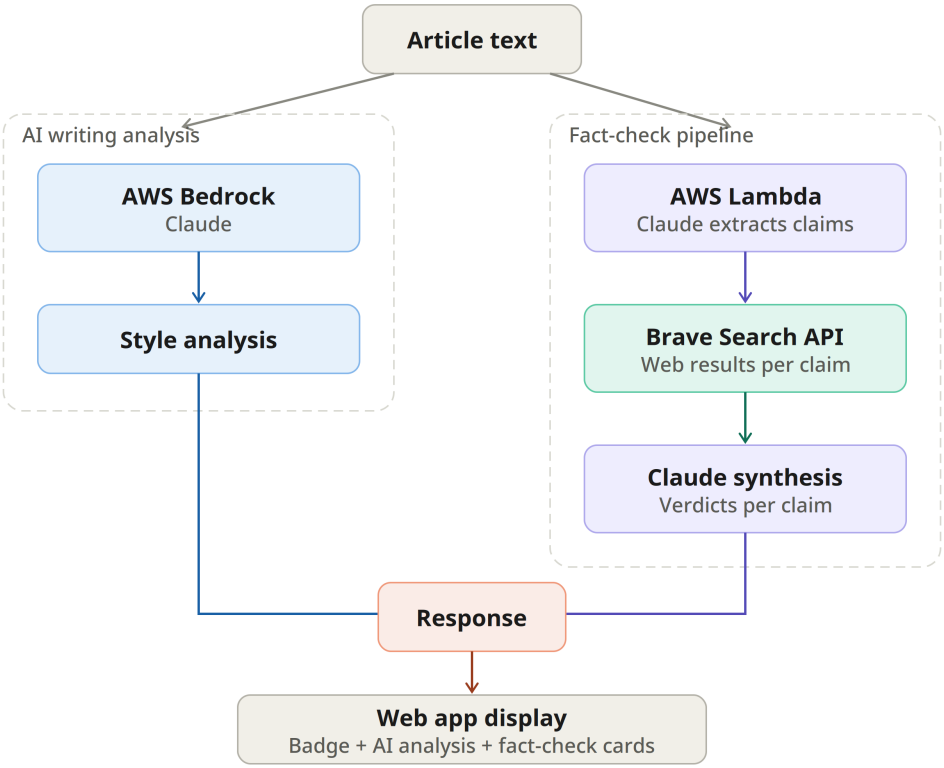


Figure 8: Parallel post-processing and response assembly

3.3 Model Performance

Table 1 summarizes the performance of all six models on the test set. We evaluated each model using accuracy, Macro F1, area under the Receiver Operating Characteristic (ROC) curve, and the Brier score (mean squared error of predicted probabilities). All models were trained and evaluated on identical data splits to ensure a fair comparison.

Model	Accuracy	Macro F1	ROC_AUC	Brier
MLP Ensemble	99.09%	0.9907	0.9993	0.0072
GRU	99.04%	0.9904	0.9995	0.0074
LSTM	98.64%	0.9864	0.9988	0.0118
BiLSTM	98.48%	0.9847	0.9983	0.0125
NB	81.20%	0.8103	0.9144	0.1816
RNN	55.97%	0.5559	0.5955	0.2430

Table 1: Model performance comparison across four metrics

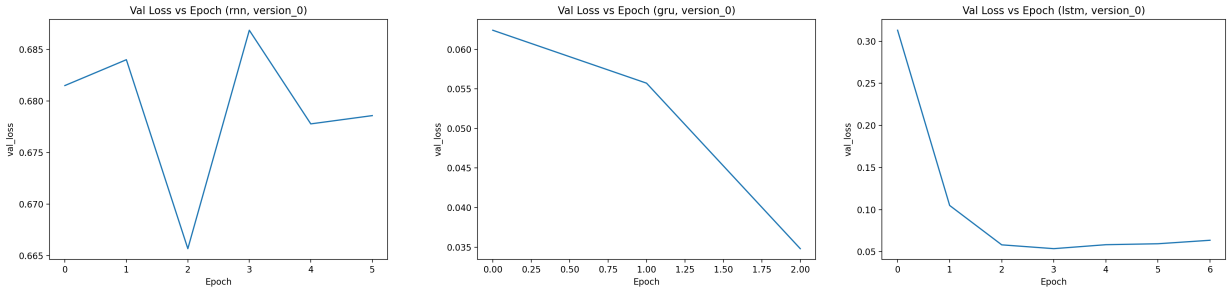
The MLP ensemble achieves the highest accuracy and Macro F1 score, though the improvement over the standalone GRU is marginal. The more significant benefit of the ensemble is its lower Brier score, which indicates more stable probability estimates. This is important since the probability is displayed to users as a credibility score. A model that is both accurate and well-calibrated gives users a more trustworthy indication of how confident the system is in its prediction.

The RNN performs only slightly better than a random guess, confirming that it cannot retain enough context for reliable text classification. The Naive Bayes model achieves a respectable 81.20% accuracy given its simplicity, but its high Brier score reveals that its probability estimates are poorly calibrated. The LSTM and BiLSTM both perform well, though the BiLSTM scores slightly lower, likely due to overfitting caused by its doubled parameter count on our dataset.

For this use case, we prioritize Brier score alongside accuracy. A false positive (classifying a real article as fake) risks the users trust in the tool, while a false negative (letting misinformation pass unchecked) defeats the purpose of the tool entirely. The ensemble provides the best balance across all four metrics, which is why it was selected as the production model.

3.4 Training Curves

The following plots show loss and accuracy over training epochs for the neural models. There are no such plots for the Naive Bayes model, as it does not train iteratively with a loss function and optimizer.



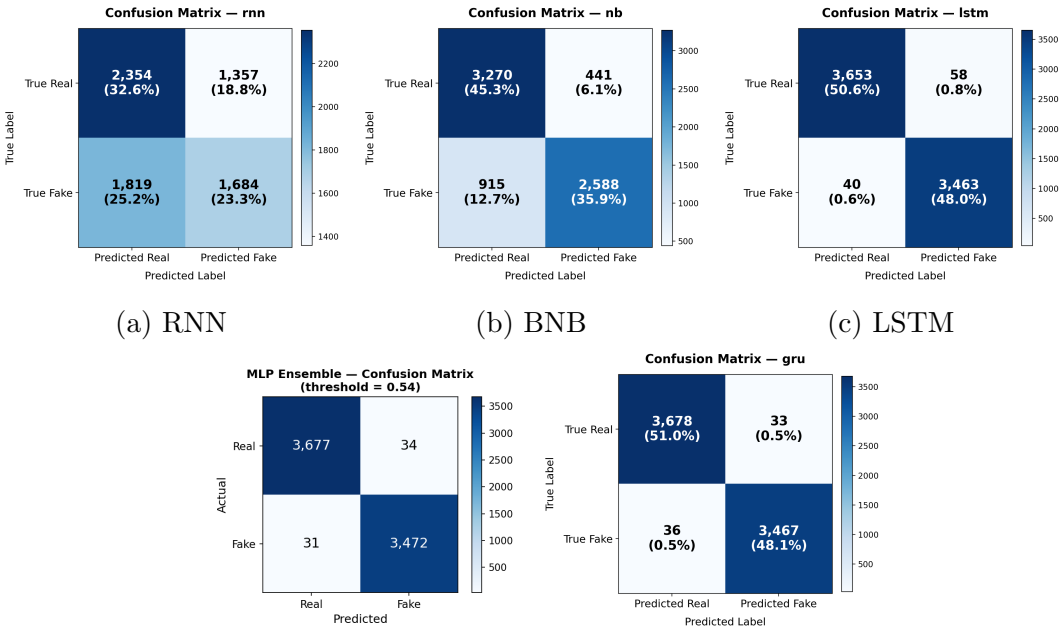
(a) RNN (b) GRU (c) LSTM

Figure 9: Validation loss over training epochs by model

The RNN loss fluctuates significantly throughout training, never settling into a stable descent. This is a consequence of the vanishing and exploding gradient problem. The GRU converges rapidly, achieving minimal loss within the first few epochs. The LSTM takes slightly longer to converge but follows a smooth, stable trajectory.

3.5 Error Analysis

Figure 10 shows the confusion matrices for each model. These illustrate the distribution of true positives, true negatives, false positives, and false negatives on the test set.



(a) RNN (b) BNB (c) LSTM

(d) Ensemble (e) GRU

Figure 10: Confusion matrices for RNN, NB, LSTM, and GRU

The RNN struggles to classify real articles correctly, performing only slightly better than a coin flip. The Naive Bayes model is considerably more capable. The LSTM and GRU models make very few errors in either direction, with no clear pattern of confusion between the two classes.

Figure 11 compares the Macro F1 scores across all models. The F1 score accounts for both precision and recall, making it a more informative metric than accuracy alone for imbalanced or borderline cases. The GRU and MLP ensemble lead, followed closely by the LSTM and BiLSTM, while the NB and RNN fall well behind.

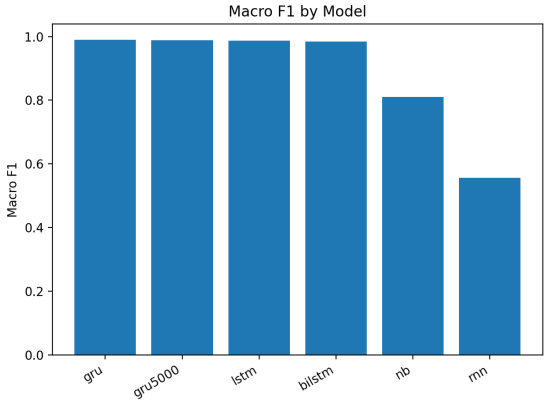


Figure 11: Macro F1 score comparison by model

3.6 Political Topic Bias

Figure 12 shows the share of misclassified articles that contain political keywords, across all models.

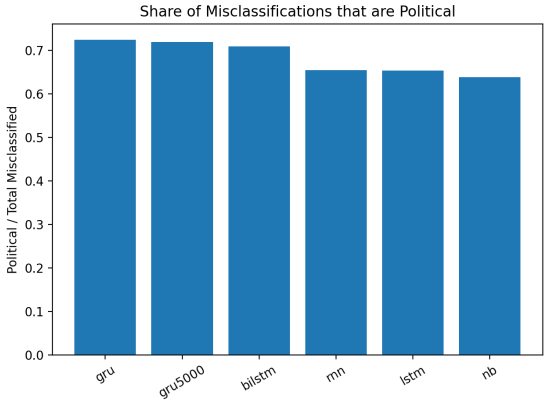


Figure 12: Share of misclassified articles containing political language

Across all models, a significant portion of misclassified articles contain political language. Approximately 73% of the GRU's errors fall into this category, with similar trends observed in the other models. This indicates a topic bias. During training, the models learned to associate political vocabulary with fake news, reflecting the composition of the WELFake dataset and its heavy weighting toward political content from 2016 – 2017. It is worth noting that this high share may be partially proportional, since political articles make up a large portion of the dataset overall (approximately 65%).

4 Discussion

The best models all achieve accuracies above 98% on the WELFake test set, but this level of performance may not generalize to other data. The dataset was compiled from four sources, most from 2016–2017, and skews heavily toward political content. Models trained on this data have likely learned stylistic patterns specific to that period. We did not test on a separate dataset due to the difficulty of finding comparable open source data, but acknowledge that accuracy would likely decrease on articles from different time periods or subject areas.

The "real" and "fake" labels in WELFake are inherited from the original source datasets. "real" articles came from editorially vetted outlets, while "fake" articles came from sources known to publish fabricated content. This binary label does not capture characteristics of misinformation such as misleading framing or opinion presented as fact. CrossCheck addresses this through the post-processing stage, where AI writing analysis and automated fact checking provide an explanation behind the classification.

The MLP ensemble was chosen for production because it achieves the highest accuracy while producing the best-calibrated probability scores, which are displayed directly to users as a credibility indicator.

5 Future Development

5.1 Adversarial Debiasing

To address the political topic bias identified in our error analysis, we plan to implement adversarial debiasing during training. This technique introduces a secondary adversary network that attempts to predict whether an article contains political content based on the

primary model's output. The adversary's success is penalized in the primary model's loss function, forcing it to learn representations that do not rely on topic-specific vocabulary to make classifications [11]. The goal is to produce a model that evaluates writing quality and factual consistency rather than reacting to the presence of political language.

5.2 Training Data Expansion

The WELFake dataset is heavily composed of political articles from 2016–2017, which limits the model's understanding of misinformation in other domains. We plan to expand the training data with more recent articles spanning news with more diverse topics. A broader training set would improve generalization to real-world usage and reduce the model's reliance on topic-specific patterns, complementing the adversarial debiasing approach described above.

5.3 Concluding Remarks

CrossCheck demonstrates that a stacked generalization ensemble, backed by AI-powered post-processing, can provide users with both a reliable credibility score and transparent reasoning behind it. The MLP ensemble was selected as the production model for its strong accuracy and well-calibrated probability estimates. Moving forward, our focus is on reducing topic bias and diversifying training data to ensure CrossCheck performs reliably across a wider range of news content.

References

[1] F. Duarte, "Amount of data created daily (2026)." Exploding Topics, Mar. 2025. Accessed: Mar. 04, 2026.

[2] S. Atske, "News platform fact sheet." Pew Research Center, Sept. 2025. Accessed: Mar. 03, 2026.

[3] S. Shahane, "Fake news classification dataset." Kaggle, 2021.

[4] S. Baladram, "Bernoulli naive bayes, explained: A visual guide with code examples for beginners," *Towards Data Science*, Aug. 2024.

[5] E. Kavlakoglu, "Naive bayes." IBM, Oct. 2021. Accessed: Mar. 04, 2026.

[6] C. Stryker, "Recurrent neural network (RNN)." IBM, Oct. 2021. Accessed: Mar. 04, 2026.

[7] Dive into Deep Learning, "10.2. gated recurrent units (GRU)." D2L.ai, 2026. Accessed: Mar. 04, 2026.

[8] J. Barnard, "Word embeddings." IBM, Dec. 2023. Accessed: Mar. 04, 2026.

[9] A. I. Naimi and L. B. Balzer, "Stacked generalization: an introduction to super learning," *European Journal of Epidemiology*, vol. 33, pp. 459–464, Apr. 2018.

[10] J. Brownlee, "Stacking ensemble machine learning with python." Machine Learning Mastery, Apr. 2020. Accessed: Mar. 04, 2026.

[11] Holistic AI, "Adversarial debiasing." Holistic AI Documentation, 2018. Accessed: Mar. 04, 2026.